WEEK 9

Q1 To perform CRUD operations and understand Replica Set configuration in MongoDB.

CODE

> use college switched
to db college
> db.createCollection("student")
{ "ok" : 1 }

INSERT

college> db.student.insertOne({name:"Alice",age:19,dept:"IT"});
college>db.student.insertMany([{name:"Bob",age:18,dept:"CSE"},{name:"John",age:20,dept:"CSE"}]);

READ

college> db.student.find()

```
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d5e6'),
  name: 'Alice',
  age: 19,
  dept: 'IT'
},
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d5e7'),
  name: 'Bob',
  age: 18,
  dept: 'CSE'
},
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d5e8'),
  name: 'John',
  age: 20,
  dept: 'CSE'
```

college> db.student.find().pretty()

OUTPUT

```
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d5e6'),
  name: 'Alice',
  age: 19,
  dept: 'IT'
}
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d5e7'),
  name: 'Bob',
  age: 18,
  dept: 'CSE'
}
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d5e8'),
  name: 'John',
  age: 20,
  dept: 'CSE'
}
```

college> db.student.find({ age: 20 })

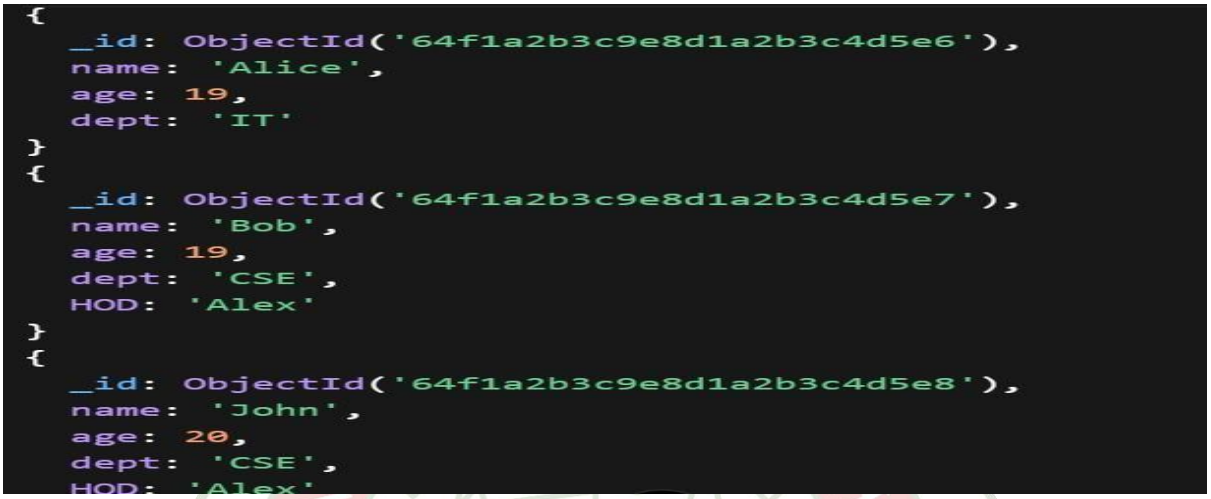
```
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d5e8'),
  name: 'John',
  age: 20,
  dept: 'CSE'
}
```

UPDATE

college> db.student.updateOne({name:"Bob"},{$set:{age:19}})

```
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d5e6'),
  name: 'Alice',
  age: 19,
  dept: 'IT'
}
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d5e7'),
  name: 'Bob',
  age: 19,
  dept: 'CSE'
}
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d5e8'),
  name: 'John',
  age: 20,
  dept: 'CSE'
}
```

college> db.student.updateMany({dept:"CSE"},{$set:{HOD:"Alex"}}) college> db.student.find().pretty()



DELETE

college> db.student.deleteOne({name:'Alice'})
college> db.student.find().pretty()

college> db.student.deleteMany({dept:"CSE"}) college>

db.student.find().pretty()

```
college> db.student.find().pretty()
```

Q1: Conditional Filtering
Insert 5 student records with fields: {name, age, dept, marks} Retrieve students:

- **age > 20** • **marks >= 75** **Code** college> db.student.insertMany([{name:"Alice",age:17,dept:"IT",Marks:70},{name:"Alex", age:21,dep t:"IT",Marks:75},{name:"Ash",age:19,dept:"ECE",Marks:80},{name:"Bob",age:1 8,dept:"CS E",Marks:85},{name:"John",age:20,dept:"CSE",Marks:90]) college> db.student.find({age:{$gt:20}})

```
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d602'),
  name: 'Alex',
  age: 21,
  dept: 'IT',
  Marks: 75
```

college> db.student.find({marks:{$gte:75}})

```
_id: ObjectId('64f1a2b3c9e8d1a2b3c4d604'),
name: 'Bob',
age: 18,
dept: 'CSE',
Marks: 85

_id: ObjectId('64f1a2b3c9e8d1a2b3c4d605'),
name: 'John',
age: 20,
dept: 'CSE',
Marks: 90

_id: ObjectId('64f1a2b3c9e8d1a2b3c4d602'),
name: 'Alex',
age: 21,
dept: 'IT',
Marks: 75

_id: ObjectId('64f1a2b3c9e8d1a2b3c4d603'),
name: 'Ash',
age: 19,
dept: 'ECE',
Marks: 80
```

Q2: Multiple Conditions (AND / OR) Find students:

• dept = "IT" AND marks > 80 • dept = "CSE" OR marks < 50

CODE

college> db.student.find({dept:"IT",Marks:{$gt:48}})

```
college> db.student.find({ dept: "IT", Marks: { $gt: 80 } })
```


college> db.student.find({$or:[{dept:"CSE"},{Marks:{$lt:45}}]})

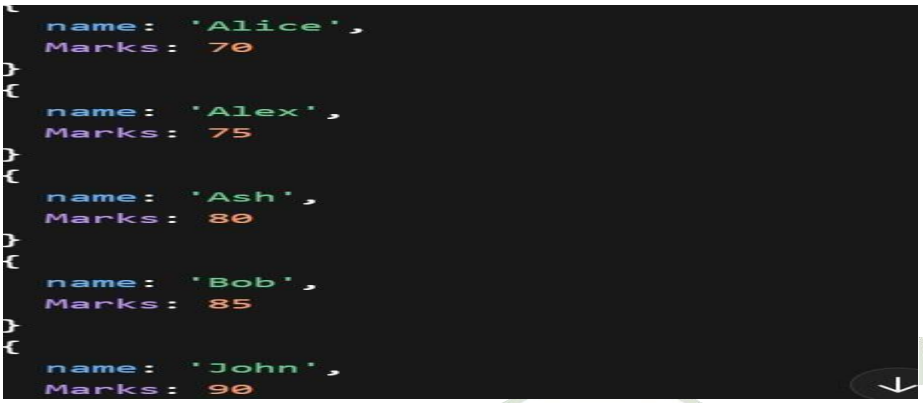
```
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d704'),
  name: 'Bob',
  age: 18,
  dept: 'CSE',
  Marks: 85
}
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d705'),
  name: 'John',
  age: 20,
  dept: 'CSE',
  Marks: 90
}
```

Q3: Projection (Selective Fields)

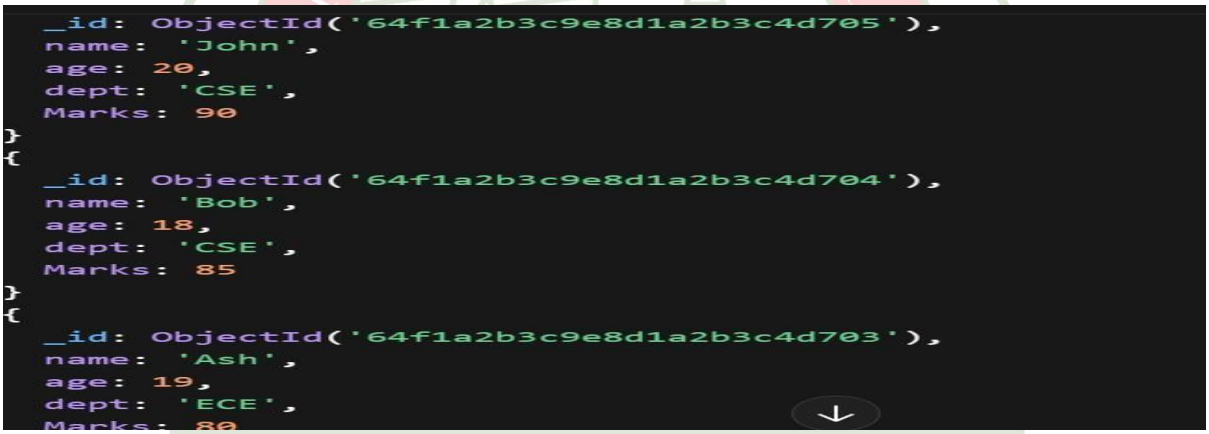
Display only name and marks, hide _id

CODE

college> db.student.find({}, { name: 1, Marks: 1, _id: 0 })



Q4: Sorting + Limiting
Display top 3 students based on marks (descending)
CODE
college> db.student.find().sort({ Marks: -1 }) .limit(3)



Q5: Pattern Matching (LIKE equivalent)
Find students whose name starts with "B"
CODE
college> db.student.find({name:{$regex :"^B"}})

Q6: Nested Condition (Analyze Level Find students:

• marks between 60 and 90 • dept not equal to "ECE" CODE

college>db.student.find({ Marks: { $gte: 60, $lte: 90 },dept: { $ne: "ECE" }})

```
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d701'),
  name: 'Alice',
  age: 17,
  dept: 'IT',
  Marks: 70

  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d702'),
  name: 'Alex',
_id: ObjectId('64f1a2b3c9e8d1a2b3c4d704'),
name: 'Bob',
age: 18,
dept: 'CSE',
Marks: 85

_id: ObjectId('64f1a2b3c9e8d1a2b3c4d705'),
name: 'John',
age: 20,
dept: 'CSE',
Marks: 90
```

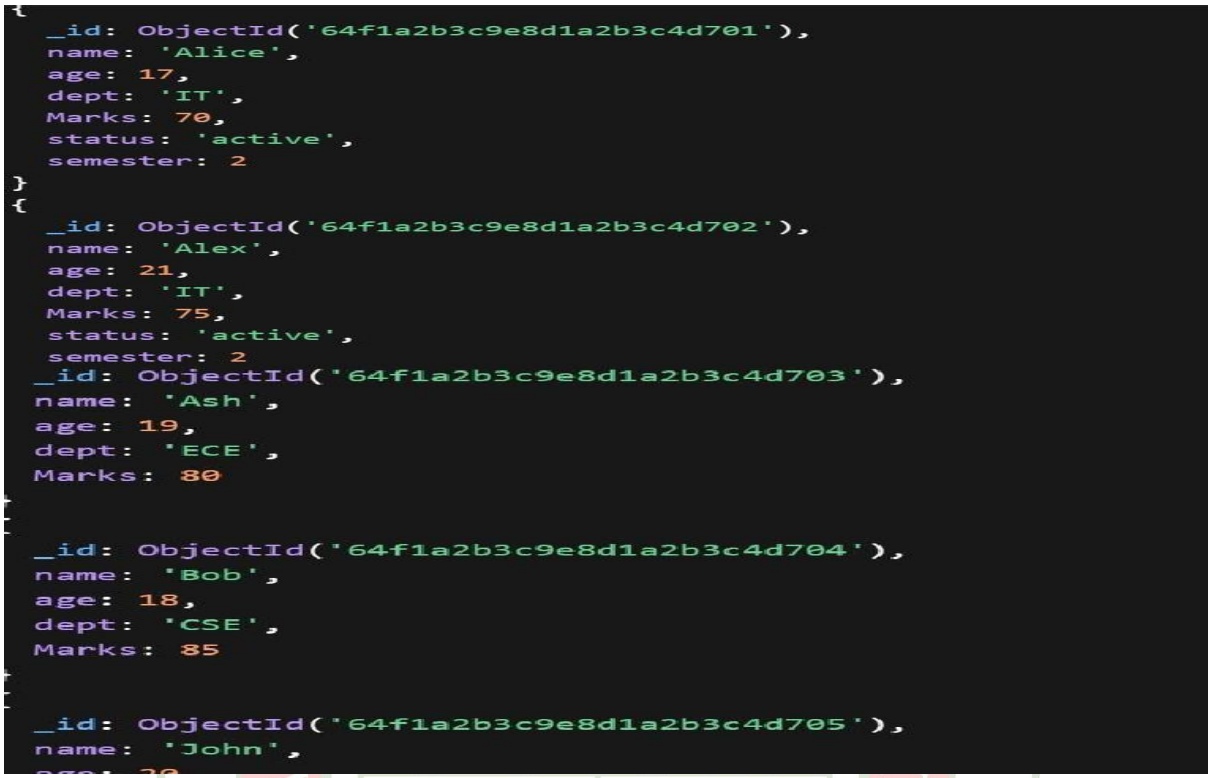
Q7: Conditional Update

Increase marks by 5 for students with marks < 50 college>db.students.updateMany(
{ marks: { $lt: 50 } }, { $inc: { marks: 5 } })

```
college> db.students.updateMany(
...     { marks: { $lt: 50 } },
...     { $inc: { marks: 5 } }
... )
{
  acknowledged: true,
  matchedCount: 0,
  modifiedCount: 0
}
```

Q8: Update with Multiple Fields For dept = "IT", add: • status = "active" • semester = 2 CODE

college> db.student.updateMany({ dept: "IT" }, { $set: { status: "active", semester: 2 } })

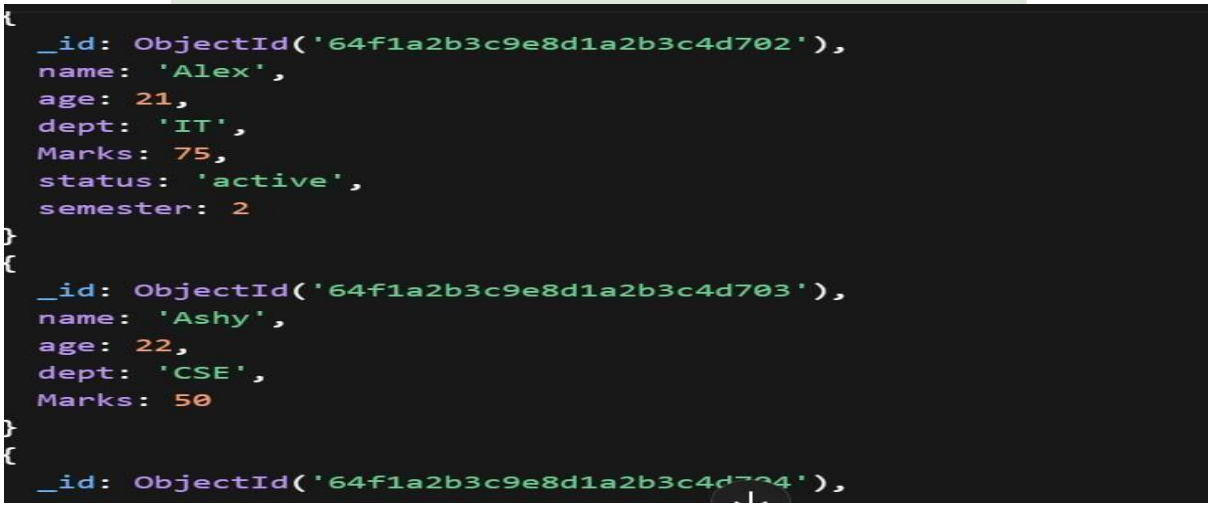


Q9: Replace Document

Replace entire document where name = "Ravi"

CODE

college> db.student.replaceOne({ name: "Ash" },{ name: "Ashy", age: 22, dept: "CSE",Marks:50 });



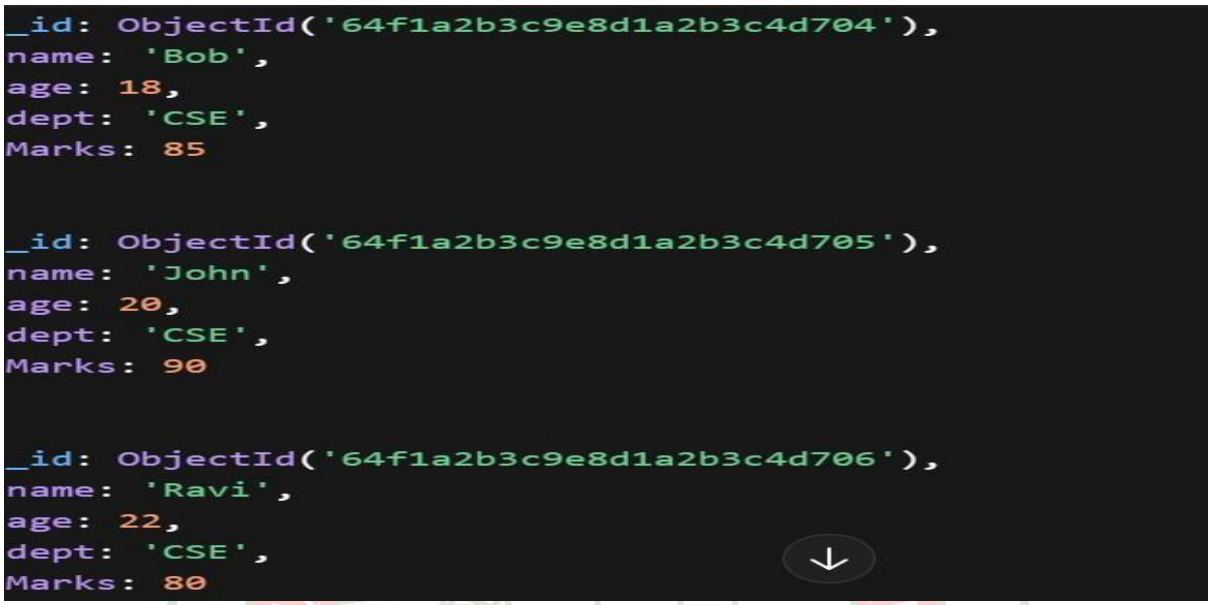
Q10: Upsert Operation (High-Level)

If student not exists → insert Else → update

CODE

college> db.student.updateOne({ name: "Ravi" },{$set: { age: 22,dept: "CSE",Marks: 80} }, { upsert: true});

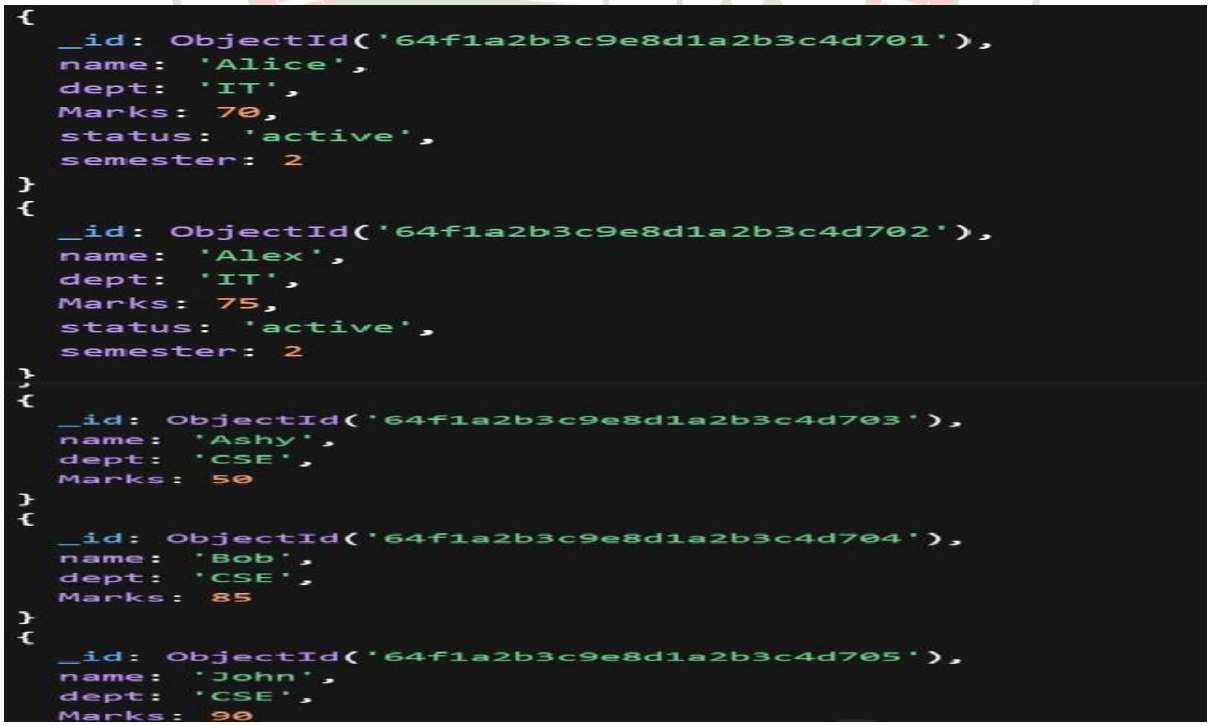
OUTPUT



Q11: Remove Field

CODE

college> db.student.updateMany({},{ $unset: { age: "" } });



Q12: Array Update (Advanced)

Insert hobbies array and add new hobby college> db.student.updateOne({ name: "Alice" }, { $push: { hobbies: "Drawing" } })

OUTPUT

```
college> db.student.find()
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d701'),
  name: 'Alice',
  dept: 'IT',
  Marks: 70,
  status: 'active',
  semester: 2,
  hobbies: ['Drawing']
}
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d702'),
  name: 'Alex',
  dept: 'IT',
  Marks: 75,
  status: 'active',
  semester: 2
}
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d703'),
  name: 'Ashy',
  dept: 'CSE',
  Marks: 50
}
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d704'),
  name: 'Bob',
  dept: 'CSE',
  Marks: 85
}
{
  _id: ObjectId('64f1a2b3c9e8d1a2b3c4d705'),
  name: 'John',
  dept: 'CSE',
  Marks: 90
```